

Part02

Q1:

A struct cannot inherit from another struct or class because structs are value types in C#. They are designed to be simple and lightweight, so C# doesn't allow inheritance between structs or from classes..

Q2:

Access modifiers control who can access a class member and where it can be accessed. public allows access from anywhere

- public allows access from anywhere
- private only within the same class
- protected within the class and derived classes
- internal within the same assembly

Q3:

Encapsulation is critical because it protects the data inside a class and controls how it can be accessed or changed.

Q4:

Constructors in structs are used to initialize the struct's fields when creating a new instance. They help make sure the struct starts with the values you need.

Q5:

Overriding methods like ToString() improves code readability by providing a clear and meaningful representation of an object instead of the default output. It makes the code easier to understand and debug.

Q6:

Structs are value types and are usually stored on the stack when they are local variables, while classes are reference types and their objects are stored on the heap. Classes are accessed through references, while structs contain their actual data.

1- A copy constructor is a constructor that creates a new object by copying the values from an existing object of the same type.

3-

An indexer allows you to access the elements of an object using an index like an array. It is useful when a class represents a collection of data.

keywords we have learnt last lecture:

- Access Modifiers: private, protected, public, internal, private protected, protected internal
- Class
- Struct
- Constructor
- Constructor Overloading
- Method Overloading
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- OOP (Object-Oriented Programming)



Salma Mahmoud · You



Computer Science Student at October 6 Univer...

5h ·



Constructors في C# - "بتعمل مش بس Object"

وأنا بتعلم C#, السطر ده وقفني شوية:

```
Student student = new Student("Salma", 21);
```

أول ما شفته، حسيت إنه بسيط: بيعمل object اسمه student.

بس لما فكرت فيها أكثر، لقيت إن اللي بيحصل مش بسيط كده.

لما تكتب new، بشكل مبسط بيحصل الآتي:

1 يتم تخصيص مساحة لل object في ال managed heap، وال fields بتاعت ال object بتبدأ بقيمها الافتراضية (0 أو null).

2 ال constructor بيشتغل ويدي ال object القيم الحقيقية.

3 ال reference بتاعت ال object دي بتتحط في المتغير student.

يعني ال constructor مش مجرد "حاجة بتشتغل لما تكتب new"، هو اللي بيساعد في تحديد ال object هيبدا حياته إزاي.

```
class Student
{
    public string Name;
    public int Age;
    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```



Add a comment...



Home



My Network



Post



Notifications



Jobs



```
{
    Name = name;
    Age = age;
}
}
```

أنواع ال Constructors الي لازم تعرفها:

◆ Parameterless - ممكن نستخدمه parameters من غير - عشان ندي قيم افتراضية.

```
public Student() => Name = "Unknown";
```

◆ Parameterized - object بياخد القيم وقت إنشاء ال -

```
public Student(string name, int age)
{
    Name = name;
    Age = age;
}
```

◆ Copy Constructor - جديد باستخدام بيانات object بيعمل - موجود object.

```
public Student(Student other)
{
    Name = other.Name;
    Age = other.Age;
}
```

◆ Static Constructor - static members يُستخدم لتهيئة ال - type. وبيتنفذ مرة واحدة لكل parameters، ومش بياخد

```
static Student()
{
    // Initialize static members
}
```



Add a comment...



Home



My Network



Post



Notifications



Jobs



◆ يمنع الكلاسات الخارجية من إنشاء - Private Constructor مباشرة instance.

```
private Student()
{
}
```

وده ممكن يكون مفيد في حالات زي Singleton أو Factory Pattern لما نكون عايزين نتحكم في طريقة إنشاء الـ objects.

كل ما بتعلم #C أكثر، بكتشف إن حتى أبسط سطر في الكود ممكن يكون وراه تفاصيل تستاهل نفهمها.

Still learning, still discovering what happens behind the code. 🚀

#CSharp #DotNet #OOP #Programming
#SoftwareDevelopment #LearningJourney

Show translation

Constructors في C#

مش بس "بتعمل Object"!

```
Student student = new Student("Salma", 21);

class Student
{
    public string Name;
    public int Age;

    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

لما تكتب **new** بيحصل الآتي بشكل مبسط:

1. Allocate: تخصيص مساحة في الـ memory.
2. Initialize: تملأ الـ fields بالقيم الافتراضية.
3. Call Constructor: تسمى الـ constructor بشكل مباشر.
4. Assign Reference: تسمى الـ reference بالـ object.

أنواع الـ Constructors

- Parameterless**
من غير parameters، ويمكن تستخدمه مشان ندي قيم افتراضية.
`public Student() => Name = "Unknown";`
- Parameterized**
يأخذ القيم وقت إنشاء الـ object.
`public Student(string name, int age) { Name = name; Age = age; }`
- Copy Constructor**
يحمل object جديد باستخدام بيانات object موجود.
`public Student(Student other) { Name = other.Name; Age = other.Age; }`
- Static Constructor**
تستخدم لتهيئة الـ static members، وعلى بتأخذ parameters، ويتنفذ مرة واحدة لكل type.
`static Student() { /* Initialize... */ }`
- Private Constructor**
يمنع الكلاسات الخارجية من إنشاء instance مباشرة مفيد في حالات زي Singleton أو Factory Pattern.
`private Student() { }`



Add a comment...



Home



My Network



Post



Notifications



Jobs



All activity

Posts

Comments

Images

Reactions

**Salma Mahmoud** · You

Computer Science Student at October 6 Univer...

4d ·  Another step forward in my .NET journey!

Happy to share that I've successfully completed C# Core Logic and Runtime Fundamentals with DotNetWithElwakeel with a score of 98%. 🚀

A great step toward strengthening my C# fundamentals and continuing my journey in .NET development.

Still learning, still improving, and looking forward to what's next! 💙

#CSharp #DotNet #SoftwareDevelopment #Learning #DotNetWithElwakeel



Home



My Network



Post



Notifications



Jobs